

MID-STATE TECHNICAL COLLEGE • IT PROGRAMS

File Management and Shell Navigation

Unit 2 • Linux (10-151-105)



What You'll Be Able to Do



By the end of this unit you'll administer a Linux system confidently from the command line. Specifically, you'll be able to:

- Describe the Filesystem Hierarchy Standard and the purpose of the key system directories.
- Navigate the filesystem fluently using absolute and relative paths and shortcuts.
- Create, copy, move, rename, view, and remove files and directories safely.
- Explain inodes and the difference between hard and symbolic links — and create both.
- Locate files with find, locate, and which, and search inside files with grep.
- Redirect input, output, and errors, and chain commands together with pipes, tee, and xargs.

Today's Roadmap

1

The Filesystem & the FHS

Where everything lives

2

Paths & Navigation

Getting around the tree

3

Working with Files

Create, copy, move, view, remove

4

Inodes & Links

What's really under a filename

5

Finding & Searching

find, locate, which, grep

6

Text, Streams & Automation

Redirection, pipes, and xargs

LESSON 1

The Filesystem & the FHS



What Is the Filesystem Hierarchy Standard?

- The FHS is a shared specification that defines which kinds of files belong in which directories.
- Nearly every distribution follows it — so configuration is always in `/etc`, user files in `/home`, logs in `/var/log`.
- That consistency is what lets you sit down at an unfamiliar Rocky or Ubuntu server and immediately know your way around.
- Software installers rely on it too, placing their files where the rest of the system expects to find them.

Why it matters

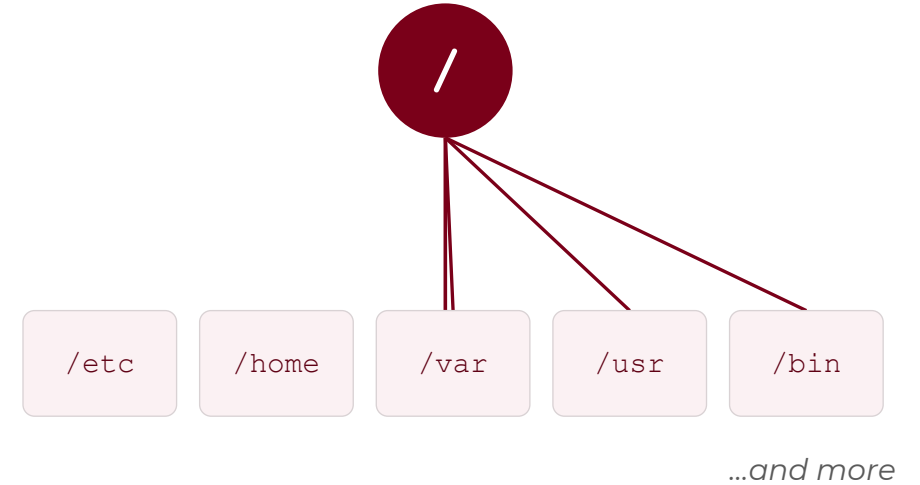
A standard layout turns “where is that file?” from a guessing game into knowledge you can carry between every Linux system you'll ever touch.

```
$ ls /
```

Lists the standard top-level directories

Everything Hangs Off the Root: /

- The single forward slash (/) is the root directory — the very top of the tree.
- It is roughly the equivalent of the C:\ drive on Windows; every other directory branches from it.
- Do not confuse / (filesystem root) with /root (the root user's home directory). They are different places.
- Absolute paths always start at / — we'll use that constantly in the next lesson.



Standard Directories (1 of 2)

Directory	What it holds
/bin	Essential command-line utilities and binaries (e.g., the program behind ls)
/boot	Files needed to boot the operating system
/dev	Device files representing hardware and software devices (e.g., /dev/sda1)
/etc	System and service configuration files
/home	Each standard user's personal home directory
/lib	Shared program libraries needed by the kernel and core utilities
/media	Mount points for removable media (USB drives, DVDs)
/mnt	Mount points for temporarily mounted filesystems

Standard Directories (2 of 2)

Directory	What it holds
/opt	Optional add-on software packages, each in its own subdirectory
/proc	Virtual filesystem exposing live kernel and process information
/root	The root user's home directory (NOT the same as /)
/sbin	System binaries used for booting and by the root user
/sys	Virtual filesystem exposing information about devices
/tmp	Temporary files, which may be cleared on shutdown
/usr	Read-only user programs and data (/usr/bin, /usr/local, /usr/share)
/var	Variable data that changes as the system runs — most importantly logs in /var/log

The Four You'll Use Most

/etc

Configuration hub — Firewall rules, SSH access, service settings
— e.g., /etc/ssh/sshd_config

/home

User files — Your personal space; the shell drops you here at login. \$HOME and ~ point to it

/root

Admin's home — The root user's home directory — not the filesystem root /

/var/log

The logs — Where the system and its services record what happens — first stop when troubleshooting

DUAL-DISTRO NOTE

Authentication events log to different files:

Rocky: /var/log/secure

Ubuntu: /var/log/auth.log

Same idea, different filename — remember it when hunting for who logged in.

Inspecting Files: stat and file

Two commands report on a file itself rather than its contents:

stat

- Displays file metadata in a readable layout
- Size, access/modify timestamps, storage details
- Syntax: `stat {filename}`

file

- Identifies what kind of file something is
- Reports type regardless of the name (PDF, JPEG, script...)
- Syntax: `file {filename}`

Tip: the name doesn't define the type. file looks inside and tells you what a file really is — useful before deciding which tool can open it.

LESSON 2

Paths & Navigation



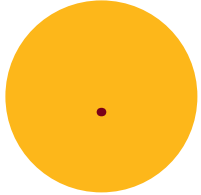
Absolute vs. Relative Paths

A path is the route through the directory tree to a file. There are two ways to write one:

	Absolute path	Relative path
Starts from	The filesystem root /	Your present working directory
Always begins with	/ (a forward slash)	A name, ., or ..
Means the same everywhere?	Yes — unambiguous from anywhere	No — depends on where you are
Example	/home/student/resources/inventory/widgets.txt	resources/inventory/widgets.txt

Best practice: use absolute paths in scripts and config files. A relative path can resolve differently depending on where it runs from — a common source of surprising bugs.

Path Shortcuts: `..` and `~`



Here — the current directory



The parent directory, one level up (stack them: `../..` climbs two)



Your home directory — expands from anywhere, e.g., `~/resources`

These work as arguments to most commands — not just `cd`. For example, `ls ..` or `cp ~/file.txt .`

Where Am I, and Moving Around

pwd

Print working directory — answers “where am I?” with the absolute path. Run it before a backup, transfer, or delete to confirm your location.

cd {path}

Change directory — moves you somewhere else, using an absolute or relative path.

```
cd /etc           # jump to the config directory
cd public/sales   # relative, down two levels
cd ..             # up one level to the parent
cd ~ (or) cd      # straight back to your home
```

LIVE DEMO

Walk the tree live on Ubuntu: `pwd` → `cd` into `/var/log` → `cd ..` → `cd ~`

Seeing Structure: tree

- tree draws the directory structure as an indented, branching diagram — far easier to read than a flat listing.
- Great for understanding how directories nest inside one another.
- Useful options: -a (include hidden files), -d (directories only), -f (show full paths), -L (limit depth).
- Note: tree isn't always installed by default — you may need to add it.

```
$ tree project
project
├── drafts
│   └── newprojects.txt
├── final
└── resources

3 directories, 1 file
```

LESSON 3

Working with Files and Directories



Listing: ls and Its Options

- On its own, ls lists the current directory; give it a path to list elsewhere.
- -a show all files, including hidden ones (names starting with a dot)
- -l long format: permissions, ownership, size, modification time
- -d the directory itself rather than its contents
- -R recurse into subdirectories

```
$ ls -al ~
```

Every item in your home directory, hidden files included, in full detail — a true workhorse combination.

Hidden ≠ secure. A leading dot just keeps a file out of the way; ls -a reveals it instantly. Hiding is tidiness, not protection.

Creating: mkdir and touch

mkdir {name}

Makes new directories. Build a layout in a few commands:

```
mkdir project
mkdir project/drafts
mkdir project/final
mkdir project/resources
```

touch {filename}

Originally updates a file's timestamp — but if the file doesn't exist, it creates an empty one. Create several at once:

```
touch a.txt b.txt c.txt
```

Confirm your work with `ls` or `tree`.

File Naming Conventions

- A name can be up to 255 bytes and contain almost any character — but a few rules and conventions keep you out of trouble.
- It cannot contain the forward slash (/) or a NULL character, and cannot be just . or .. (those are reserved).
- By convention, separate words with a hyphen (-) or underscore (_), not a space.
- A space makes a name ambiguous on the command line — audit file.txt can be read as two items.

Good: `audit_file.txt` `audit-file.txt`

Risky: `audit file.txt` (the space bites)

Copy, Move, and Rename

Both commands take the source first and the destination second.

cp {source} {dest}

Duplicates a file, leaving the original in place. Good habit: back up a config before editing it.

```
cp /etc/ssh/sshd_config \  
  /root/sshd_config.old
```

mv {source} {dest}

Moves a file — and doubles as the rename command when source and destination are in the same directory.

```
mv sales.txt marketing.txt
```

There is no separate “rename” command in Linux — mv is it. This surprises almost everyone at first.

LIVE DEMO

Copy a file, then mv it to a new name; show ls before and after

Displaying File Contents: cat

- cat (short for concatenate) prints a file's contents — ideal for short files.
- Combine echo with redirection to create content quickly, then verify with cat.
- For long files, reach for less, head, or tail instead (coming up next).

```
$ echo "Q1 goals" > drafts/newprojects.txt
$ cat drafts/newprojects.txt
Q1 goals
```

Why “concatenate”?

cat can join several files together into one stream of output — cat a.txt b.txt. Displaying a single file is just the most common use of that ability.

Removing Files and Directories

rmmdir {name}

Removes an empty directory. Refuses if anything is still inside.

rm {filename}

Deletes files. Add -R for a non-empty directory tree; -i prompts, -f forces with no prompt.



Caution: there is no undo

There is no trash can at the command line. Once `rm` removes a file, it's gone. Be especially wary of `rm -fR`, which deletes an entire tree without asking. Run `pwd` and `ls` first — cheap insurance — and double-check the path before pressing Enter.

Viewing Long Files

Command	What it does	Best for
less	Pages through a file; scroll up AND down; press q to quit	Reading or searching long files
more	Older pager; scrolls downward only	Quick one-way paging
head	Shows the first 10 lines (-n to change the count)	Comments/version notes atop config files
tail	Shows the last 10 lines; -f follows the file live	Newest entries in a log

```
$ ls /etc | less           # page through long output
$ tail -f /var/log/syslog  # watch a log update in real time
```

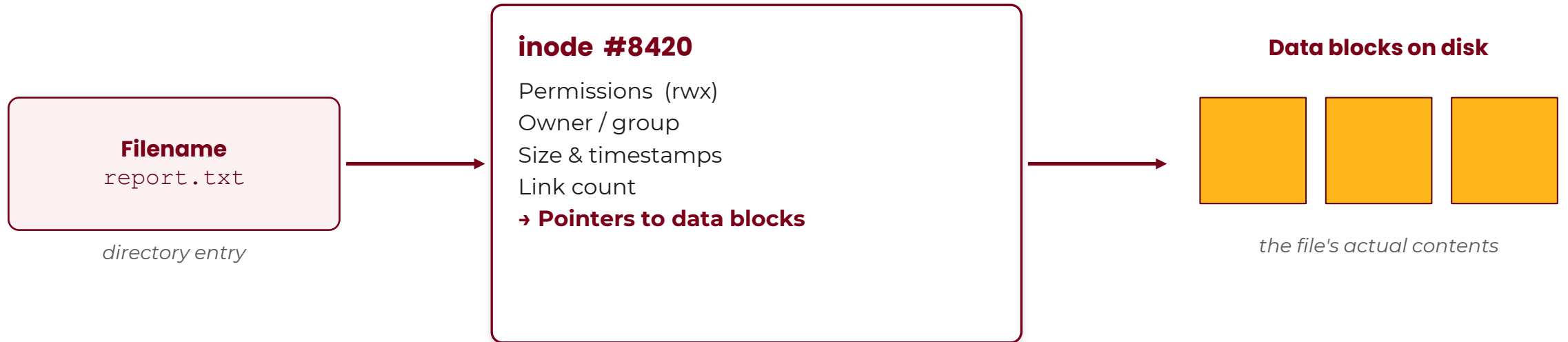
LESSON 4

Inodes and Links



What Is an Inode?

Every file and directory is described by an inode — an object that stores the file's metadata and points to where its data actually lives on disk. The filename is just a label that leads to the inode.



The Inode Table

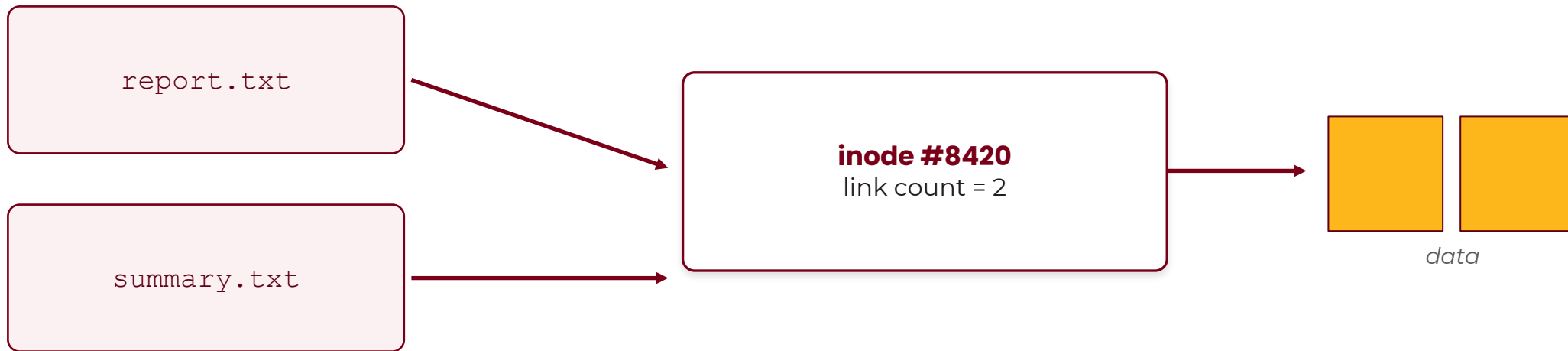
- Each inode has a unique number. When the system accesses a file, it looks that number up in the inode table.
- Think of the inode table as the master index mapping inode numbers to their data locations.
- Some filesystems fix the inode count at creation time (based on size); others, like XFS, allocate them dynamically.
- You can run out of inodes even with free space left — millions of tiny files is the classic cause.

```
$ ls -i  
8420 report.txt  
9100 link.txt
```

`ls -i` shows each file's inode number — the quickest way to tell whether two names share one inode.

Hard Links: Two Names, One Inode

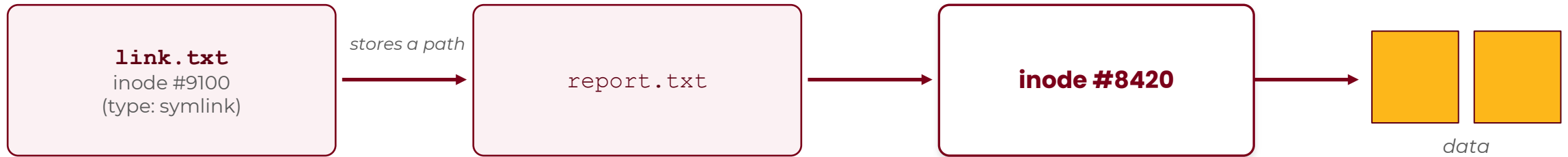
A hard link gives the same data a second name. Both names point to the very same inode — they are equal, not original-and-copy.



Delete one name and the data survives — the link count just drops to 1. The data is freed only when the count reaches 0. Limits: hard links can't cross filesystems or link directories. Create with `ln {target} {link}`.

Symbolic Links: a Pointer to a Name

A symbolic (soft) link is its own small file with its own inode. It stores a path that points at another name — not directly at the data.



Delete report.txt and the symlink dangles — it now points to a name that's gone. But symlinks can cross filesystems and link directories, which hard links cannot. Create with `ln -s {target} {link}`.

Hard vs. Symbolic Links

	Hard link	Symbolic (soft) link
Created with	ln {target} {link}	ln -s {target} {link}
Points to	The same inode (the data itself)	A path / name — a different inode
If original is deleted	Data remains — link count drops	Link breaks (dangles)
Cross filesystems?	No	Yes
Link a directory?	No	Yes
Shares inode number?	Yes (ls -li proves it)	No — has its own

LIVE DEMO

Create both link types, run `ls -li`, then delete the target and show what survives

Creating Links: ln

Syntax: `ln {options} {target} {link-name}`

```
ln /business-docs/current-projects.txt ~/projects.txt # hard link
ln -s /business-docs/current-projects.txt ~/projects.txt # symbolic link
ls -li ~/projects.txt # inspect it
```

- `-s` make a symbolic link instead of a hard link
- `-f` remove an existing destination first
- `-i` prompt before overwriting a destination
- `-v` print each name as it is linked

Why link at all?

A file buried deep in the tree can be reached through a short, convenient name in a user's home directory — without copying the data.

LESSON 5

Finding and Searching



find: a Live Search

- find performs a real-time search of the filesystem — always accurate, though slower on huge trees.
- Hugely flexible: match by name, type, permissions, size, modification date, owner, and more.
- -name (by name) -type f / -type d (file or directory) -perm (permissions)

```
find /home/student/projects -name business-plan.txt
find /etc -perm 777          # files with rwx for everyone (audit!)
find / -type d -name logs    # all directories named 'logs'
```

Syntax: find {where to search} {options} {search criteria}

locate and updatedb

- locate searches a prebuilt index database instead of the live filesystem — so it's very fast.
- The catch: results are only as current as the index. A brand-new file won't appear until the index updates.
- updatedb rebuilds that index (from /etc/updatedb.conf; database at /var/lib/mlocate/mlocate.db).
- -i ignores case.

```
sudo updatedb          # refresh the index first
locate -i reports.txt
```

find vs. locate, and which

	find	locate
How it searches	Live scan of the filesystem	Prebuilt index database
Speed	Slower on large trees	Very fast
Accuracy	Always current	Only as fresh as last updatedb
Best for	Precise, criteria-based searches	Quick “where is that file?” lookups

which {command}

Finds a different kind of thing: the full path of the program that runs when you type a command, by searching your PATH. which cat returns /bin/cat — invaluable when the “wrong” version of a command seems to run.

grep: Searching Inside Files

- Where find and locate find files, grep finds text inside files — printing every line that matches a pattern.
- -i ignore case (matches error, Error, ERROR alike)
- -n show the line number of each match
- -R search recursively through a directory and its subdirectories
- -E extended regular expressions (replaces the deprecated egrep)

```
$ grep error /var/log/messages
```

Shows every line of the log that contains “error.”

grep + Pipes: the Real Power

grep shines when fed another command's output through a pipe (|), acting as a filter.

```
$ ls /etc | grep -i net          # items in /etc whose names contain 'net'
$ grep jdeng /etc/passwd       # does this user account exist?
$ ps aux | grep ssh            # is the SSH process running?
$ grep -R "TODO" ~/project     # find TODOs across a whole tree
```

LIVE DEMO

Pipe ls and ps output into grep on Ubuntu; add -i and -n to show the difference

LESSON 6

Text, Streams, and Automation



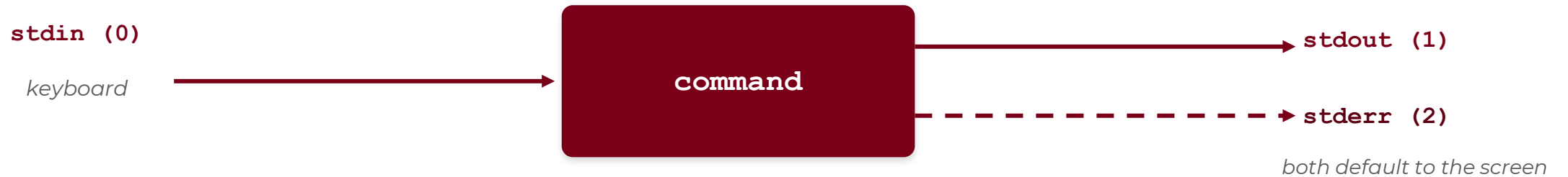
The Text-Processing Toolkit

A family of small, focused commands transform, count, and reorganize text — each doing one job well, and most happy to be piped together. Recognize what each is for; you'll master options with practice.

Command	What it does
echo / printf	Print text; printf adds precise formatting control (newlines, columns)
wc	Count lines (-l), words (-w), characters (-m), or bytes (-c)
sort	Order lines; -n numeric, -r reverse, -k by field
uniq	Drop ADJACENT duplicate lines — almost always used as sort file uniq
cut / paste	Extract columns from lines / merge files side by side into columns
tr	Translate or substitute characters in a stream (e.g., case change)
diff / sdiff	Compare two files; sdiff shows the differences side by side
sed / awk	Stream editor (find-and-replace) / field-processing mini-language

Text Streams: `stdin`, `stdout`, `stderr`

Every command reads from and writes to three streams. Redirection lets you reroute any of them.



The numbers matter: 1 = standard output, 2 = standard error. You'll use them to redirect each stream independently.

Redirection Operators

Operator	What it does
>	Send stdout to a file, OVERWRITING it (ls > files.txt)
>>	Send stdout to a file, APPENDING to the end (ls >> files.txt)
<	Read stdin from a file instead of the keyboard
2> 2>>	Redirect standard error to a file (overwrite / append)
&>	Redirect BOTH stdout and stderr to one file
 	Pipe — use one command's output as the next command's input
<<	Here document — supply inline input until a marker line (e.g., EOF)

Overwrite vs. Append — and Errors



> overwrites, >> appends. Reaching for the wrong one is how people wipe out a file they meant to add to. When in doubt, use >> .

```
ls > files.txt           # create/overwrite files.txt with the listing
ls >> files.txt           # add the listing to the end instead
cat missing 2> err.txt    # capture only the error message
make &> build.log         # capture normal output AND errors together
```

Remember: 1 is the output stream, 2 is the error stream — that's why error redirection uses 2> .

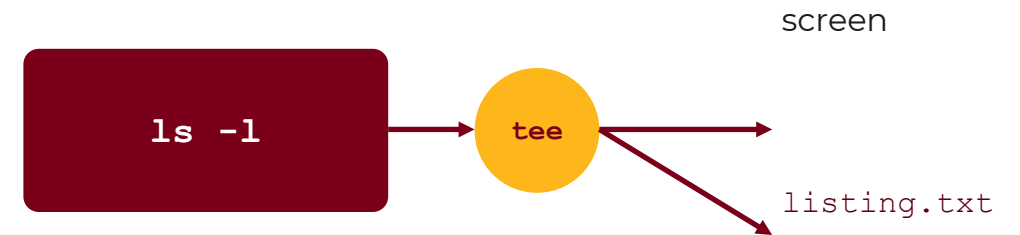
Chaining Commands Together

Operator	Behavior
	Pipe — feed one command's output into the next (ls -l grep audit)
;	Run commands in sequence, regardless of success
&&	Run the next command ONLY IF the previous one succeeded
	Run the next command ONLY IF the previous one failed
&	Run a command in the background as a job; keep using the shell
!	Negate an expression (“not this”)

These connect the small tools into real workflows. && is especially common: do step two only if step one worked.

tee: Split the Stream

- Normally output goes one place. tee sends it TWO ways at once — to the screen AND to a file.
- Perfect when you want to watch a command's output and keep a copy of it.
- Add -a to append to the file instead of overwriting it.



```
$ ls -l | tee listing.txt
```

xargs: Turning Output into Arguments

Some commands read from stdin (like `grep`). Others — `rm`, `mkdir`, `touch`, `cp` — expect their input as command-line ARGUMENTS and simply ignore stdin. That's the problem `xargs` solves.

Doesn't work

```
find . -name '*.pdf' | rm
```

`rm` ignores stdin entirely, so it deletes nothing — it just complains it got no filenames.

Works

```
find . -name '*.pdf' | xargs rm
```

`xargs` reads the names from stdin and rebuilds them as arguments to `rm` — so `rm` actually receives the files.

In one line: `xargs` is an adapter that converts a stream of items on stdin into arguments for a command.

xargs: a Pipeline Broken Down

1 find runs

```
find /projects -type f -name "*.pdf"
```

2 it produces a list

```
file1.pdf  file2.pdf  file3.pdf
```

3 the pipe carries it

```
| sends that list to xargs on stdin
```

4 xargs builds a command

```
rm file1.pdf file2.pdf file3.pdf
```

5 the command runs once

```
all three PDFs deleted in a single rm call
```

By default xargs batches as many arguments as the system allows into as few command runs as possible — efficient for large lists.

xargs: Options and the Whitespace Gotcha

- -I {} place each item at a chosen spot in the command (great when the file isn't the last argument)
- -n {N} run the command with at most N arguments at a time
- -p prompt for confirmation before each run; -t print each command before running it
- -0 read items separated by NULL instead of whitespace — pair with find -print0



The whitespace trap: a filename with a space looks like two items to plain xargs. The safe pattern uses NULL separators end to end.

```
find . -name '*.log' -print0 | xargs -0 rm          # safe with spaces
find . -name '*.log' | xargs -I{} mv {} /archive  # place {} explicitly
```

xargs: When to Reach for It

- Running a command on every result of a search — the classic `find ... | xargs ...` pattern.
- Acting on a list of items from a file or another command (one command instead of a manual loop).
- Bulk operations: delete, move, copy, change ownership across many files at once.
- Feeding arguments to commands that don't read stdin on their own (`rm`, `mkdir`, `cp`, `kill...`).

Good to know

`find` also has its own `-exec` and `-delete` for the same job, and shell loops can do it too. We'll build on these ideas when we get to scripting later in the course — for now, recognize `xargs` as the go-to way to turn a list into action.

This Week's Lab



NETLAB+ Lab: File Management and Shell Navigation

- Starts from the WK02_FILESYSTEM known-good snapshot — reset anytime if you get stuck.
- Navigate the FHS and build/reorganize a directory structure on BOTH Rocky 10 and Ubuntu 24.04.
- Create and remove files, then work with hard and symbolic links and confirm them with `ls -li`.
- Locate files with `find` and `locate`; search inside files with `grep`.
- Capture and filter command output using redirection and pipes.

Remember: the pod reverts at the end of your reservation, so capture your deliverables during the session per the lab instructions.

Security Focus: Filesystem Visibility



We don't cover permissions formally until Unit 4 — but security starts the moment you can see the filesystem. A lot of real-world information disclosure begins with nothing more than layout.

- Use `ls -l` to see who owns a file and who can read it — even before you can change those settings.
- Know that `/etc` holds sensitive configuration and `/var/log` can reveal who did what and when.
- Hidden files (leading dot) are hidden for tidiness, not security — `ls -a` shows them instantly.
- Be careful where output lands: redirecting sensitive data into a world-readable file quietly exposes it.
- Treat `find` as an auditing tool — later you'll hunt for files readable or writable by everyone.

Why it matters: you can't protect what you can't see. Noticing ownership, location, and visibility now makes the hardening work in later units feel obvious.

Key Takeaways

- The FHS is a shared map — learn it once and navigate any Linux system.
- Absolute paths start at / and mean the same everywhere; relative paths depend on where you are.
- A filename leads to an inode, which holds the metadata and points to the data.
- Hard link = another name for the same inode; symbolic link = a pointer to a name (and can break).
- find searches live, locate searches an index, which finds a command's binary, grep searches inside files.
- Redirection (>, >>, 2>, &>) and pipes (|) compose small tools; xargs turns a list into arguments.

MID-STATE TECHNICAL COLLEGE · IT PROGRAMS

Questions?

Unit 2 · File Management and Shell Navigation

